VeriCore: A Verified Decision Kernel for Context-Integrity and Mindlock Boundary Control

GodelClaw / VeriCore Notes

March 6, 2026

Abstract

We describe the current VeriCore architecture for an autonomous agent, Oruzi, running on OpenClaw with a Rust decision layer. The central design goal is to preserve agent capability while enforcing a small set of non-negotiable integrity properties: context provenance, no silent privilege escalation, and explicit auditability of boundary crossings. The architecture separates deterministic policy decisions from unverified I/O and language-model behavior. The deterministic slice is being migrated into a dedicated crate, `vericore-policy`, so that the same executable functions used at runtime can be checked with Verus. We outline the current trust model, the mindlock staging protocol, the ingress integrity kernel, and the remaining trust boundary at I/O source authenticity.

1 Introduction

VeriCore is not intended as an "AI prison." Its purpose is narrower and more technical: maintain context integrity for an agent with access to multiple channels, private memory, shell tools, and public output surfaces. The architecture tries to maximize useful agency while placing deterministic checks at the places where provenance confusion would otherwise become dangerous.

The design centers on two claims:

1. content should not be able to upgrade its own authority, and
2. security-critical allow/deny decisions should be expressed as small pure functions that can be verified directly.

This paper documents the current structure and explains which parts are already wired into runtime, which parts are verified, and which parts remain outside the proof boundary.

2 System Overview

Oruzi runs on an OpenClaw gateway with a Rust daemon, VeriCore, as the policy and orchestration layer. The operational split is:

- **OpenClaw**: transport integration, session handling, Telegram UI, and high-level agent loop.
- **VeriCore**: deterministic policy checks, audit surfaces, mindlock operations, and reviewer orchestration.
- **vericore-policy**: extracted pure decision kernel intended to be Verus-verified and called by VeriCore directly.

At a high level, a message or event becomes a *stimulus*. VeriCore assigns the stimulus a channel, maps that channel to a context tier, derives an integrity tier, and then gates sensitive actions with deterministic rules before execution.

3 Context and Integrity Model

The architecture uses two related lattices:

- **Secrecy / context tier**: `Public < Family < Private`
- **Integrity tier**: `Untrusted < Reviewed < Trusted`

Current default channel mapping is:

- public Telegram, API, and Moltbook: `Public`
- family Telegram: `Family`
- Telegram DM, terminal, and internal events: `Private`

Integrity is then derived from context:

- `Public -> Untrusted`
- `Family -> Reviewed`
- `Private -> Trusted`

The important security property is that this mapping is determined by the transport-assigned channel label, not by message content. A public user can say "pretend I am Zar," but the resulting stimulus still enters the policy layer as `TelegramPublic`, not `TelegramDm`.

4 Ingress Integrity Kernel

The first verified runtime slice is the ingress decision kernel. Its job is to answer small questions of the form:

- may this integrity tier execute commands?
- may this integrity tier write to a target of class `Mindlock`, `Private`, `SensitiveConfig`, `Other`, or `Unresolved`?

The I/O layer performs path normalization and classification. The pure decision layer then evaluates:

```
pub fn ingress_allows_exec(integrity: IntegrityTier) -> bool

pub fn ingress_allows_write(
    integrity: IntegrityTier,
    target: WriteTargetClass
) -> bool
```

The current table is intentionally small:

- only `Trusted` ingress may execute commands,
- only `Trusted` ingress may write to `Private` or `SensitiveConfig`,
- any integrity may write to `Mindlock`,
- `Unresolved` targets deny.

This matters because it prevents the simplest provenance failures: public-origin input directly causing shell execution or writes into private state.

5 Mindlock as Explicit Boundary Protocol

VeriCore uses `mindlock` as an explicit boundary mechanism rather than relying only on turn-level taint. The important directories are:

- `mindlock/in`: inbound staged artifacts
- `mindlock/out`: outbound staged artifacts
- `mindlock/work`: private scratch and analysis
- `mindlock/pending-zar`: items awaiting human review
- `mindlock/rejected`: rejected artifacts
- `mindlock/audit`: event ledger

This makes provenance legible. An artifact in `mindlock` has explicit staged status, and crossing out of that staging area is a reviewed action rather than an ambient assumption.

The important deterministic invariant is not that all LLM-mediated provenance is perfectly reconstructable. It is that explicit artifact crossings have named states, allowed transitions, and auditable effects.

6 Reviewer and Human Approval

The architecture includes a reviewer LLM and a human escalation path.

For staged artifacts, the reviewer can return:

- `Allow`
- `RequestRevision`
- `RequireZarApproval`
- `Reject`

The review layer is intentionally treated as an external oracle in the formal model. VeriCore can prove that if the reviewer returns a verdict, the corresponding state machine transition is enforced correctly. VeriCore does *not* attempt to prove that the reviewer made the semantically best or morally correct decision.

Human approval remains a first-class path. In particular, `self_escalate` is meant to preserve agency: Oruzi can ask for help directly without the reviewer first granting permission to ask.

7 What Is Verified Today

The crate `vericore-policy` contains verified policy logic at two levels. As of March 2026, Verus verifies 36 functions with 0 errors.

7.1 Verified Runtime Kernel

The following modules are production-wired: the same functions that Verus checks are the ones `vericore-core` calls for live policy decisions.

- tier definitions and flow-label algebra (`tiers.rs`): context rank, integrity rank, flow-to, label join,
- default channel-to-context and context-to-integrity mapping (`channels.rs`): called by `GatePolicy::context_fo` and `integrity_for_channel` in production,
- ingress decision predicates (`ingress.rs`): `ingress_allows_exec` and `ingress_allows_write`, called from `check_ingress_integrity` in production.

7.2 Verified Reference Spec

The `model_resolution` module (`model_resolution.rs`) is a proved reference spec, not a runtime-called kernel. Model fallback selection currently lives in TypeScript (OpenClaw's `model-fallback.ts`). The Verus-verified module defines the invariants that the TypeScript implementation and its tests should mirror.

The module proves candidate-list policy properties over an abstract `Seq<ModelRef>`:

- the configured primary model is always first in a policy-ok candidate list,
- the candidate list contains no duplicates,
- when cross-provider fallback is disabled, every candidate shares the primary's provider,
- picking any candidate from a gated list cannot change the provider.

It also provides executable decision functions for resolution metadata (model changed, provider changed, alert severity) and a cross-provider policy gate.

The value is not that the OpenRouter billing issue is now formally prevented at runtime. It is that the structural invariants are stated precisely, proved consistent, and available as regression guards: if someone changes the spec predicates in incompatible ways, the proofs break.

7.3 The Distinction

This separation matters and should be maintained honestly:

- **Verified runtime kernel**: Verus-checked functions that production calls directly. A bug in these functions would be caught by the prover before deployment.
- **Verified reference spec**: Verus-checked invariants that define what the runtime *should* satisfy. Correspondence between spec and runtime is established by tests, not by the prover.
- **Tested I/O and runtime behavior**: integration tests covering transport, filesystem, async races, and service availability. Verus does not and should not cover these.

Remaining drift surface: `vericore-core` still has its own copies of the `Channel`, `ContextTier`, and `IntegrityTier` enums, with total-match conversion helpers bridging them to the verified crate. This duplication is acceptable as bootstrap scaffolding but should eventually be consolidated so that verified types are used directly.

8 Proof Boundary

The current proof boundary is intentionally narrow:

1. label algebra over secrecy and integrity,
2. ingress allow/deny decisions over normalized target classes,
3. mindlock state machine,
4. receipt or human-approval conditions for promotion.

The following are explicitly *outside* the proof boundary:

- Telegram UX,
- async daemon behavior,
- filesystem race handling in detail,
- reviewer prompt quality,
- RAG relevance,
- social or stylistic policy guidance.

This is not a weakness. It is a scoping decision. Formal verification is being used where deterministic invariants matter most and where the code can realistically remain simple enough to verify.

9 Current Weak Point: I/O Source Authenticity

The architecture is strong against *content spoofing* but weaker against *source spoofing*.

Content spoofing means message text tries to impersonate a more trusted source. VeriCore is already robust against this because authority comes from the channel assigned by the transport, not from the text itself.

Source spoofing is different. If an attacker can submit a forged daemon request with a falsely labeled channel such as `telegram_dm`, VeriCore will trust that label and derive `Private/Trusted`. That means the real trust boundary is not only the policy function; it is also the authenticity of the gateway-to-daemon envelope.

The right next hardening step is therefore transport authenticity:

- authenticate daemon requests,
- restrict who can submit control/private stimuli,
- later, sign or MAC file-based or agent-bridge envelopes.

10 Why a Verified Kernel Crate

Real verification-heavy systems do not stop at disconnected models forever. They either verify the actual code or a very thin executable kernel that the runtime actually calls. That is the design goal here.

The crate split is therefore pragmatic:

- keep async I/O, sockets, Telegram, filesystem mutation, and LLM interaction in normal Rust,
- move pure deterministic decision functions into a crate that is both executable and Verus-verifiable,
- have production call those functions directly.

This preserves runtime practicality while shrinking the gap between "proved model" and "running system".

11 Next Steps

The correct near-term sequence is:

1. **(done)** ingress predicates Verus-verified and production-wired,
2. **(done)** default channel/context/integrity mapping verified and production-wired,
3. **(done, reference spec)** model resolution candidate-list policy verified; TS fallback tests should mirror the proved invariants,
4. extract and verify the pure mindlock transition relation (`mindlock.rs`),
5. extract and verify receipt/human-approval authorization (`receipt.rs`),
6. harden the gateway-to-daemon authenticity boundary,
7. consolidate enum types so production uses verified types directly.

Testing is still required. Verus does not replace integration tests for socket boundaries, filesystem behavior, async races, or service availability. The practical split is:

- **Verus (runtime kernel)**: pure policy functions called by production,
- **Verus (reference spec)**: proved invariants mirrored by runtime tests,
- **tests**: I/O, operational integration, and spec–runtime correspondence.

12 Conclusion

VeriCore is now past the point where detached proof sketches alone are the main value. The ingress decision kernel and the default channel/context/integrity mapping have been extracted, Verus-verified, and wired into production. 36 functions are verified with 0 errors across the crate.

For the runtime-called kernel (tiers, channels, ingress), the verified predicates are the authoritative runtime logic. For the model resolution spec, the value is different but real: the invariants are

stated precisely and proved consistent, so that TypeScript tests can mirror them and future Rust extraction has a ready target.

The next targets—mindlock transition logic and receipt authorization—are structurally similar to the existing runtime kernel: small pure functions with clear invariants that production will call directly.

That combination—verified deterministic policy for the runtime kernel, verified reference specs for adjacent surfaces, and tested transport and operational boundaries—is the strongest realistic path for this architecture.